VSP TECHVERSE
An Institute of AI & Robotics

Data Structures in Python for AI & Robotics



Python Lists · Tuples · Sets · Dictionaries — adapted for AI & Robotics

VSP TECHVERSE
An Institute of AI & Robotics



Python Lists

Lists are used to store multiple items in a single variable.

TOPICS IN THIS SECTION

1. Python Lists
2. Access List Items
3. Change List Items
4. Add List Items
5. Remove List Items
6. Loop Lists
7. List Comprehension
8. Sort Lists
9. Copy Lists
10. Join Lists
11. List Methods

Python Lists

example.py

```
sensors = ["lidar", "camera", "imu"]
print(sensors)
print(len(sensors))
print(type(sensors))
```

OUTPUT

```
['lidar', 'camera', 'imu']
3
<class 'list'>
```

KEY POINTS

- Lists hold several items in a single variable
- Items keep their order, are indexed from zero, and new items go to the end
- Lists can be changed, may repeat values, and can mix data types

KEY NOTE

Lists are used to store multiple items in a single variable.

Access List Items

example.py

```
sensors = ["lidar", "camera", "imu"]
print(sensors[1])
print(sensors[-1])
```

OUTPUT

```
camera
imu
```

KEY POINTS

- Each list item has a numbered position you can use to retrieve it
- Counting starts at 0, so the first item is at position 0
- Negative positions count backward from the end; -1 is the last item

KEY NOTE

The first item has index 0.

Change List Items

example.py

```
sensors = ["lidar", "camera", "imu"]
sensors[1] = "sonar"
print(sensors)

sensors.insert(2, "gps")
print(sensors)
```

OUTPUT

```
['lidar', 'sonar', 'imu']
['lidar', 'sonar', 'gps', 'imu']
```

KEY POINTS

- Replace a single item by targeting its position number
- Assign a new list to a slice of positions to swap several items at once
- insert() adds an item without replacing any

KEY NOTE

To change the value of a specific item, refer to the index number.

Add List Items

example.py

```
sensors = ["lidar", "camera", "imu"]
sensors.append("radar")
print(sensors)

more = ["gps", "sonar"]
sensors.extend(more)
print(sensors)
```

OUTPUT

```
['lidar', 'camera', 'imu', 'radar']
['lidar', 'camera', 'imu', 'radar', 'gps', 'sonar']
```

KEY POINTS

- append() adds an item to the end of the list
- insert() places an item at a specified index
- extend() adds the elements of another iterable to the end

KEY NOTE

To add an item to the end of the list, use the append() method.

Remove List Items

example.py

```
sensors = ["lidar", "camera", "imu", "gps"]
sensors.remove("camera")
print(sensors)

sensors.pop()
print(sensors)
```

OUTPUT

```
['lidar', 'imu', 'gps']
['lidar', 'imu']
```

KEY POINTS

- remove() deletes a list item by its value
- If the value appears more than once, only the first match is removed
- pop() and del remove items by index; clear() empties the list

KEY NOTE

The remove() method removes the specified item.

Loop Lists

example.py

```
sensors = ["lidar", "camera", "imu"]
for x in sensors:
  print(x)

for i in range(len(sensors)):
  print(sensors[i])
```

OUTPUT

```
lidar
camera
imu
lidar
camera
imu
```

KEY POINTS

- A for loop steps through each item, one at a time
- Loop by index with range() and len()
- A while loop with a manually increased index also works

KEY NOTE

You can loop through the list items by using a for loop.

List Comprehension

example.py

```
sensors = ["lidar", "camera", "imu", "gps", "sonar"]

newlist = [x for x in sensors if "a" in x]

print(newlist)
```

OUTPUT

```
['lidar', 'camera', 'sonar']
```

KEY POINTS

- Creates a new list from an existing one with less code than a loop
- The original list is left unchanged
- An optional condition filters items

KEY NOTE

List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

Sort Lists

example.py

```
sensors = ["radar", "lidar", "imu", "sonar",
           "camera"]
sensors.sort()
print(sensors)

sensors.sort(reverse = True)
print(sensors)
```

OUTPUT

```
['camera', 'imu', 'lidar', 'radar', 'sonar']
['sonar', 'radar', 'lidar', 'imu', 'camera']
```

KEY POINTS

- Python list objects have a sort() method that sorts ascending by default
- Passing reverse = True sorts the list in descending order
- Adding key = str.lower gives a case-insensitive sort

KEY NOTE

List objects have a sort() method that orders items in ascending order by default.

Copy Lists

example.py

```
sensors = ["lidar", "camera", "imu"]
mylist = sensors.copy()
print(mylist)

mylist = list(sensors)
print(mylist)
```

OUTPUT

```
['lidar', 'camera', 'imu']
['lidar', 'camera', 'imu']
```

KEY POINTS

- list2 = list1 does not make a real copy — both names point to the same list
- copy() creates an independent copy
- list() and the [:] slice are other ways to copy

KEY NOTE

You cannot copy a list simply by typing list2 = list1.

Join Lists

example.py

```
list1 = ["lidar", "camera", "imu"]
list2 = [1, 2, 3]

list3 = list1 + list2
print(list3)

list1.extend(list2)
print(list1)
```

OUTPUT

```
['lidar', 'camera', 'imu', 1, 2, 3]
['lidar', 'camera', 'imu', 1, 2, 3]
```

KEY POINTS

- The + operator combines two lists
- A loop can append each item from one list into another
- extend() adds the elements of one list to the end of another

KEY NOTE

There are several ways to join, or concatenate, two or more lists in Python.

List Methods

example.py

```
sensors = ["lidar", "camera", "lidar"]

x = sensors.count("lidar")

print(x)
```

OUTPUT

```
2
```

KEY POINTS

- Python has a set of built-in methods you can use on lists
- The page lists them in a table with a short description for each
- Example: count() returns the number of elements with the specified value

METHODS

append · clear · copy · count · extend · index · insert · pop · remove · reverse · sort



Python Tuples

Tuples are used to store multiple items in a single variable.

TOPICS IN THIS SECTION

1. Python Tuples
2. Access Tuples
3. Update Tuples
4. Unpack Tuples
5. Loop Tuples
6. Join Tuples
7. Tuple Methods

Python Tuples

example.py

```
joints = ("base", "elbow", "wrist")
print(joints)
print(len(joints))
```

OUTPUT

```
('base', 'elbow', 'wrist')
3
```

KEY POINTS

- Tuples are one of four built-in collection types, with lists, sets and dictionaries
- Tuples are ordered and unchangeable, and can contain duplicate values
- Items are indexed from [0]; len() returns the number of items

KEY NOTE

Tuples are used to store multiple items in a single variable.

Access Tuples

example.py

```
joints = ("base", "elbow", "wrist")
print(joints[1])
print(joints[-1])
```

OUTPUT

```
elbow
wrist
```

KEY POINTS

- Retrieve items by position number in square brackets
- Counting begins at 0, so the first item is index 0
- Negative numbers count from the end; -1 is the last item

KEY NOTE

Negative indexing means start from the end.

Update Tuples

example.py

```
x = ("base", "elbow", "wrist")
y = list(x)
y[1] = "shoulder"
x = tuple(y)

print(x)
```

OUTPUT

```
('base', 'shoulder', 'wrist')
```

KEY POINTS

- Tuples can't be edited after creation, so their values stay fixed
- To change one, convert it to a list, edit it, then convert it back
- The whole tuple can be deleted with del

KEY NOTE

Tuples are unchangeable, meaning that you cannot change, add, or remove items once the tuple is created.

Unpack Tuples

example.py

```
joints = ("base", "elbow", "wrist")

(j1, j2, j3) = joints

print(j1)
print(j2)
print(j3)
```

OUTPUT

```
base
elbow
wrist
```

KEY POINTS

- Creating a tuple with assigned values is known as packing
- Unpacking pulls a tuple's values out into separate variables
- An asterisk can gather leftover values into a list

KEY NOTE

This is called "packing" a tuple.

Loop Tuples

example.py

```
joints = ("base", "elbow", "wrist")
for x in joints:
  print(x)

for i in range(len(joints)):
  print(joints[i])
```

OUTPUT

```
base
elbow
wrist
base
elbow
wrist
```

KEY POINTS

- A for loop steps through every item in a tuple
- range() and len() let you loop by index number
- A while loop can also walk through items with an index counter

KEY NOTE

You can loop through the tuple items by using a for loop.

Join Tuples

example.py

```
tuple1 = ("x", "y" , "z")
tuple2 = (10, 20, 90)

tuple3 = tuple1 + tuple2
print(tuple3)

joints = ("base", "elbow", "wrist")
mytuple = joints * 2
print(mytuple)
```

OUTPUT

```
('x', 'y', 'z', 10, 20, 90)
('base', 'elbow', 'wrist', 'base', 'elbow', 'wrist')
```

KEY POINTS

- Combine two or more tuples with the + operator
- Repeat a tuple's contents with the * operator
- The result is a new tuple

KEY NOTE

To join two or more tuples you can use the + operator.

Tuple Methods

example.py

```
ids = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)

x = ids.count(5)

print(x)
```

OUTPUT

```
2
```

KEY POINTS

- Python has two built-in methods you can use on tuples
- They are listed in a summary table
- Example: count() returns how many times a value appears

METHODS

count · index



Python Sets

Sets are used to store multiple items in a single variable.

TOPICS IN THIS SECTION

1. Python Sets
2. Access Set Items
3. Add Set Items
4. Remove Set Items
5. Loop Sets
6. Join Sets
7. Frozenset
8. Set Methods

Python Sets

example.py

```
sensors = {"lidar", "camera", "imu"}
print(sensors)
```

OUTPUT

```
{'imu', 'lidar', 'camera'}
```

KEY POINTS

- Sets hold multiple items in a single variable
- One of four built-in collection types, with lists, tuples and dictionaries
- Sets ignore duplicate values

KEY NOTE

Sets are unordered, so you cannot be sure in which order the items will appear.

Access Set Items

example.py

```
sensors = {"lidar", "camera", "imu"}
for x in sensors:
  print(x)

print("camera" in sensors)
```

OUTPUT

```
imu
lidar
camera
True
```

KEY POINTS

- Set items can't be reached by position or by key
- A for loop can step through every value in a set
- in checks whether a value is present; not in checks it's absent

KEY NOTE

You cannot access items in a set by referring to an index or a key.

Add Set Items

example.py

```
sensors = {"lidar", "camera", "imu"}
sensors.add("radar")
print(sensors)

more = {"sonar", "gps", "encoder"}
sensors.update(more)
print(sensors)
```

OUTPUT

```
{'imu', 'radar', 'lidar', 'camera'}
{'sonar', 'encoder', 'imu', 'lidar', 'gps', 'camera',
'radar'}
```

KEY POINTS

- Existing items can't be changed, but new items can be added
- add() inserts a single item
- update() adds items from another set or any iterable, such as a list

KEY NOTE

Once a set is created, you cannot change its items, but you can add new items.

Remove Set Items

example.py

```
sensors = {"lidar", "camera", "imu"}
sensors.remove("camera")
print(sensors)

sensors.discard("camera")
print(sensors)
```

OUTPUT

```
{'imu', 'lidar'}
{'imu', 'lidar'}
```

KEY POINTS

- remove() and discard() both drop an item
- remove() raises an error when the item is absent; discard() does not
- pop() deletes an arbitrary element and returns it

KEY NOTE

If the item to remove does not exist, remove() will raise an error.

Loop Sets

example.py

```
sensors = {"lidar", "camera", "imu"}
for x in sensors:
  print(x)
```

OUTPUT

```
imu
lidar
camera
```

KEY POINTS

- Loop through a set one item at a time with a for loop
- The loop variable (here x) takes each value in turn
- Order is not fixed, because sets are unordered

KEY NOTE

You can loop through the set items by using a for loop.

Join Sets

example.py

```
set1 = {"lidar", "camera", "imu"}
set2 = {"imu", "gps", "sonar"}

set3 = set1.union(set2)
print(set3)
```

OUTPUT

```
{'sonar', 'imu', 'lidar', 'gps', 'camera'}
```

KEY POINTS

- union() and update() combine all items from both sets, leaving out duplicates
- intersection(), difference() and symmetric_difference() keep different subsets
- union() returns a new set

KEY NOTE

The union() method returns a new set with all items from both sets.

Frozenset

example.py

```
mylist = ["lidar", "camera", "imu"]
x = frozenset(mylist)

print(x)
print(type(x))
```

OUTPUT

```
frozenset({'imu', 'lidar', 'camera'})
<class 'frozenset'>
```

KEY POINTS

- frozenset() creates an immutable version of a set
- It accepts any iterable: a list, tuple or set
- Once created, you can't add or remove items

KEY NOTE

frozenset() is a built-in Python function that creates an immutable version of a set.

Set Methods

example.py

```
x = {"lidar", "camera", "imu"}
y = {"gps", "sonar", "imu"}

z = x.union(y)

print(z)
```

OUTPUT

```
{'sonar', 'imu', 'lidar', 'gps', 'camera'}
```

KEY POINTS

- Python has a set of built-in methods you can use on sets
- They cover adding, removing, copying, clearing and comparing sets
- Many have a shortcut operator, such as - for difference()



METHODS

add · clear · copy · difference · difference_update · discard · intersection · intersection_update · isdisjoint · issubset · issuperset · pop · remove · symmetric_difference · symmetric_difference_update · union · update



Python Dictionaries

Dictionaries are used to store data values in key:value pairs.

TOPICS IN THIS SECTION

1. Python Dictionaries
2. Access Items
3. Change Items
4. Add Items
5. Remove Items
6. Loop Dictionaries
7. Copy Dictionaries
8. Nested Dictionaries
9. Dictionary Methods

Python Dictionaries

example.py

```
robot = {
  "name": "Rover-2",
  "type": "Explorer",
  "battery": 64
}
print(robot)
```

OUTPUT

```
{'name': 'Rover-2', 'type': 'Explorer', 'battery': 64}
```

KEY POINTS

- Dictionaries store values as pairs of a key and a value
- You can look up a value by its key name
- Dictionaries are changeable: add, change or remove items after creating them

KEY NOTE

Dictionaries are used to store data values in key:value pairs.

Access Items

example.py

```
robot = {
  "name": "Rover-2",
  "type": "Explorer",
  "battery": 64
}
x = robot["type"]
print(x)

x = robot.get("type")
print(x)
```

OUTPUT

```
Explorer
Explorer
```

KEY POINTS

- Read a value by writing the key name in square brackets
- get() returns the same value as square-bracket access
- keys(), values() and items() return views that update when the dictionary changes

KEY NOTE

You can access the items of a dictionary by referring to its key name, inside square brackets.

Change Items

example.py

```
robot = {
  "name": "Rover-2",
  "type": "Explorer",
  "battery": 64
}
robot["battery"] = 80
print(robot)
```

OUTPUT

```
{'name': 'Rover-2', 'type': 'Explorer', 'battery': 80}
```

KEY POINTS

- Assign a new value to an existing key to change that item
- update() can change a value by passing a dictionary of key-value pairs
- The example changes the robot's battery level from 64 to 80

KEY NOTE

You can change the value of a specific item by referring to its key name.

Add Items

example.py

```
robot = {
  "name": "Rover-2",
  "type": "Explorer",
  "battery": 64
}
robot["status"] = "online"
print(robot)
```

OUTPUT

```
{'name': 'Rover-2', 'type': 'Explorer', 'battery': 64,
'status': 'online'}
```

KEY POINTS

- Assign a value to a new key to add that entry
- update() adds an item if it's missing and changes it if it exists
- update() accepts a dictionary or an iterable of key-value pairs

KEY NOTE

Adding an item to the dictionary is done by using a new index key and assigning a value to it.

Remove Items

example.py

```
robot = {
  "name": "Rover-2",
  "type": "Explorer",
  "battery": 64
}
robot.pop("type")
print(robot)

robot.popitem()
print(robot)
```

OUTPUT

```
{'name': 'Rover-2', 'battery': 64}
{'name': 'Rover-2'}
```

KEY POINTS

- pop() removes the item with the specified key name
- popitem() removes the last inserted item
- clear() empties the dictionary

KEY NOTE

The pop() method removes the item with the specified key name.

Loop Dictionaries

example.py

```
robot = {
  "name": "Rover-2",
  "type": "Explorer",
  "battery": 64
}
for x, y in robot.items():
  print(x, y)
```

OUTPUT

```
name Rover-2
type Explorer
battery 64
```

KEY POINTS

- A for loop can loop through a dictionary
- By default, looping returns the keys
- values(), keys() and items() return values, keys or both

KEY NOTE

When looping through a dictionary, the return value are the keys of the dictionary, but there are methods to return the values as well.

Copy Dictionaries

example.py

```
robot = {
  "name": "Rover-2",
  "type": "Explorer",
  "battery": 64
}
mydict = robot.copy()
print(mydict)
```

OUTPUT

```
{'name': 'Rover-2', 'type': 'Explorer', 'battery': 64}
```

KEY POINTS

- Assigning one dictionary to a new name does not create an independent copy
- Edits to the original also appear under the other name
- copy() and dict() make a real copy

KEY NOTE

You cannot copy a dictionary simply by typing dict2 = dict1.

Nested Dictionaries

example.py

```
fleet = {
  "arm" : {"name" : "Arm-1", "battery" : 82},
  "rover" : {"name" : "Rover-2", "battery" : 64}
}

print(fleet["rover"]["name"])
for x, obj in fleet.items():
  print(x)
```

OUTPUT

```
Rover-2
arm
rover
```

KEY POINTS

- A dictionary can hold other dictionaries inside it
- Reach inner values by chaining keys
- items() loops through outer and inner keys

KEY NOTE

A dictionary can contain dictionaries, this is called nested dictionaries.

Dictionary Methods

example.py

```
robot = {
  "name": "Rover-2",
  "type": "Explorer",
  "battery": 64
}

x = robot.get("type")

print(x)
```

OUTPUT

```
Explorer
```

KEY POINTS

- Python has a set of built-in methods you can use on dictionaries
- get() and pop() return or remove values by key
- update() and setdefault() add or change key-value pairs

METHODS

clear · copy · fromkeys · get · items · keys · pop · popitem · setdefault · update · values

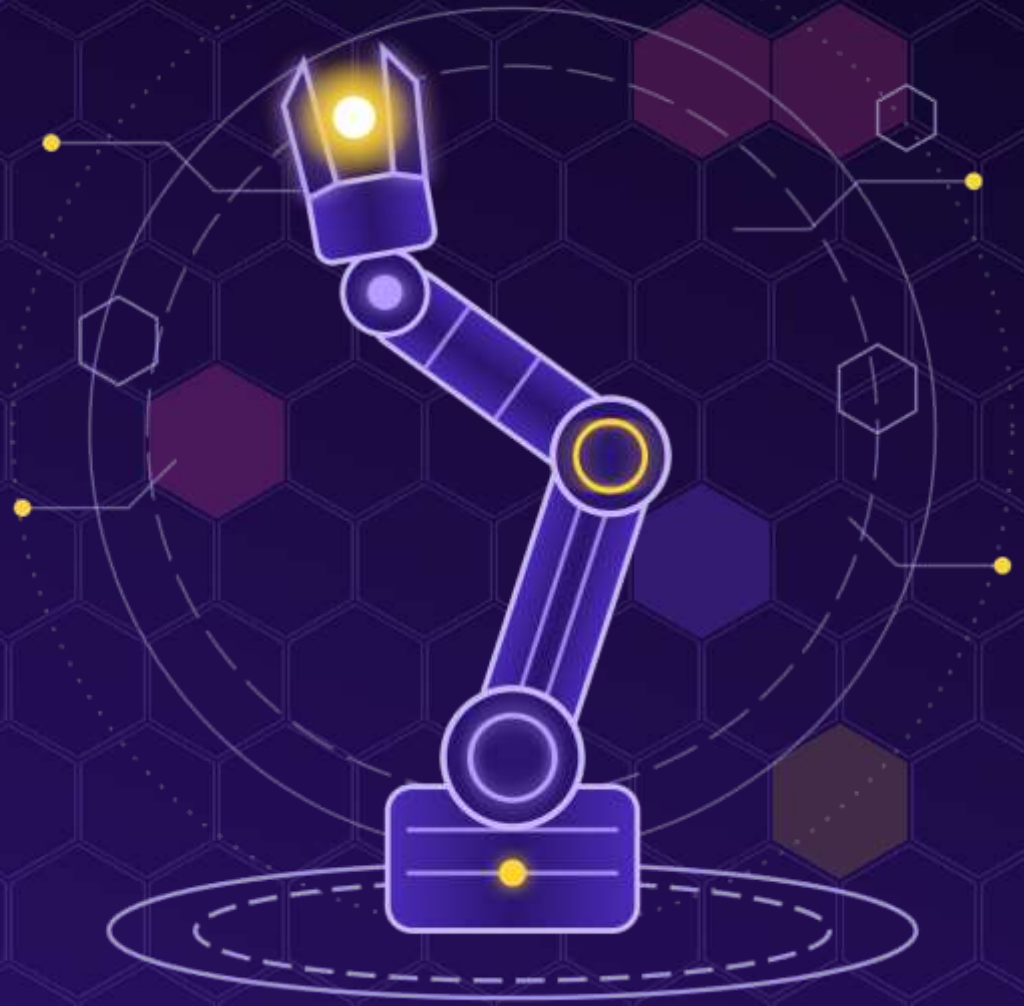
SUMMARY

List vs Tuple vs Set vs Dictionary

Feature	List	Tuple	Set	Dictionary
Brackets	`[ ]`	`( )`	`{ }`	`{ key: value }`
Ordered	Yes	Yes	No	Yes (by insertion)
Changeable	Yes	No	Add / remove only	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys must be unique
Access items	`Index: sensors[0]`	`Index: joints[0]`	Loop or "in" check	`Key: robot["name"]`
Robotics example	`["lidar", "camera"]`	`("base", "elbow")`	`{"lidar", "imu"}`	`{"name": "Rover-2"}`
Best used for	Readings that change	Fixed config values	Unique sensor IDs	Labelled robot data

Thank You

Concepts, examples adapted for AI & Robotics



VSP TECHVERSE
An Institute of AI & Robotics